

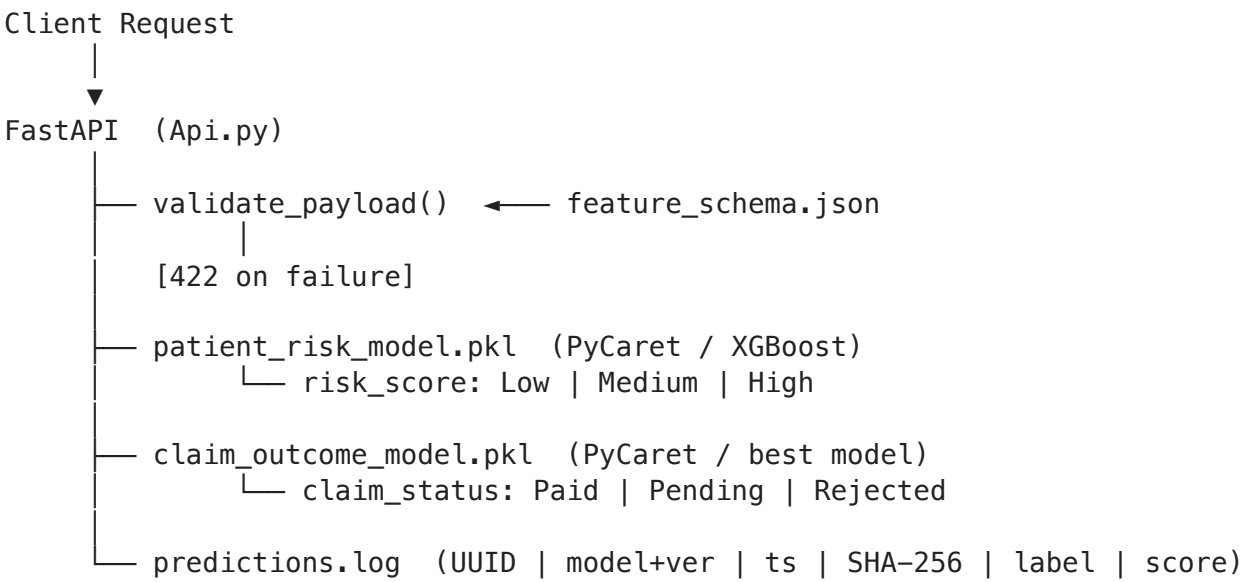
Phase 6 — Notebook 3: Governance and Compliance Document

Document purpose: Establish the governance framework for the Healthcare ML system — covering system boundaries, known limitations, audit trail design, retraining strategy, and compliance obligations.

Field	Value
Document version	1.0
Date	2026-05-01
Models covered	patient_risk_model v1.0 , claim_outcome_model v1.0
Serving system	FastAPI (Phase 5) — Api.py
Status	Active

Section 1: System Overview

1.1 Architecture



1.2 Model Versions

Model	Version	Algorithm	Training Rows	Target
patient_risk_model	1.0	XGBoost (tuned)	25,000	risk_score (3-class)
claim_outcome_model	1.0	PyCaret best (auto-tuned)	25,000	claim_status (3-class)

1.3 Data Flow

- 1. Raw data ingested from `patients.csv` , `visits.csv` , `billing.csv`
- 2. Features engineered in Phase 2 → `model_table.csv`
- 3. Models trained in Phase 3 with time-based 80/20 split + SMOTE balancing
- 4. Models serialised to `Model Files/*.pkl` and served via Phase 5 API
- 5. Every prediction logged to `logs/predictions.log`

Section 2: Known Limitations

2.1 Patient Risk Model

Limitation	Impact	Mitigation
Only 3 features (<code>chronic_flag</code> , <code>gender</code> , <code>visit_frequency</code>)	Risk score is a coarse proxy — does not capture diagnosis severity, comorbidities, or socioeconomic factors	Flag for Phase 2 feature expansion if additional data becomes available
Binary <code>chronic_flag</code> collapses multiple conditions	Patients with 1 chronic condition and 3 are treated identically	Replace with condition count or severity score in v2.0
SMOTE applied on training split only	Generated synthetic minority samples may not represent real distribution shifts	Monitor High-risk precision in production

2.2 Claim Outcome Model

Limitation	Impact	Mitigation
Trained on synthetic/simulated data	Distribution may not match real hospital claims	Retrain on real-world data before any production deployment
<code>payment_days</code> may not be available at prediction time	Feature requires ground truth from billing cycle	Impute with provider average at inference time; flag in API docs
<code>lag_time</code> can be negative in training data	Indicates billing before visit — data quality issue in source	Enforce <code>min_value: 0</code> in schema v1.1 after investigation
PyCaret auto-selection is non-deterministic	Model identity may change across retraining runs without version pinning	Fix model type in v2.0 (<code>model_type</code> param in PyCaret)

2.3 General System Limitations

- **No real-time retraining:** Models are static files. Concept drift is not automatically detected and corrected.
- **No online learning:** The system cannot learn from new predictions without an explicit retrain-and-redeploy cycle.
- **Single-region deployment:** No redundancy or load balancing documented.
- **Dataset size:** 25,000 rows is small for healthcare AI. Minority class sizes (High-risk: 5,034; Rejected: 3,797) may cause overfitting.

Section 3: Assumptions

The following assumptions underpin both models. If any are violated in production, model performance may degrade significantly.

#	Assumption	Rationale	Monitoring Signal
A1	Input distributions are stationary (IID)	ML models assume training and serving distributions match	DataDriftPreset — Notebook 2
A2	Label definitions are stable	<code>risk_score</code> and <code>claim_status</code> categories do not change meaning	TargetDriftPreset — Notebook 2
A3	Feature engineering pipeline is identical at serving time	Features in <code>Api.py</code> match those in Phase 2 <code>Build Features.ipynb</code>	Schema versioning in <code>feature_schema.json</code>
A4	<code>chronic_flag</code> is a reliable binary indicator	Sourced from structured patient records, not free-text NLP	Data source audit
A5	<code>billed_amount</code> is pre-negotiation (gross charge)	Affects model calibration for Rejected claims	Billing system changelog
A6	City categories are exhaustive	No new cities will appear in production data	Unseen category validation check (Notebook 1)
A7	Insurance providers are stable	No new providers or mergers will change the allowed set	<code>allowed_values</code> in schema; HTTP 422 on unknown provider

Section 4: Audit Log Governance

4.1 Log Schema

Every prediction appends one line to `logs/predictions.log` :

```
<timestamp> INFO PREDICTION | id=<UUID> | model=<name> v<version> | ts=<UTC ISO-8601> | hash=<SHA-256> | label=<class> | score=<float>
```

Field	Example	Purpose
<code>id</code>	<code>56fc596d-3f25-4d86-9131-0ff84bb9f30f</code>	Unique trace ID for each prediction
<code>model</code>	<code>patient_risk_model v1.0</code>	Identifies which model version was used
<code>ts</code>	<code>2026-05-01T14:48:18.796775+00:00</code>	UTC timestamp for chronological ordering
<code>hash</code>	<code>aecbd702...</code>	SHA-256 of the feature dict — proves which input produced the output
<code>label</code>	<code>Medium</code>	Predicted class
<code>score</code>	<code>0.3521</code>	Model confidence (max of softmax probabilities)

4.2 Privacy Design

Raw patient features are **never** written to the log. The SHA-256 hash:

- Is deterministic (same input always produces the same hash)
- Is one-way (cannot reverse the hash to recover patient data)
- Allows correlation: if the same payload is resent, the hash will match — enabling duplicate detection

4.3 Log Retention Policy (Recommended)

Period	Action
0–90 days	Hot storage (local disk) — fast access for incident investigation

Period	Action
91–365 days	Cold storage (compressed archive) — compliance audit trail
> 365 days	Delete or anonymise per applicable data protection policy

4.4 Model Version Tracking

Model version strings (`PATIENT_RISK_VERSION` , `CLAIM_OUTCOME_VERSION`) are hardcoded in `Api.py` and written to every log entry. When a model is retrained:

- 1. Increment the version string (e.g., `1.0` → `1.1`)
- 2. Replace the `.pkl` file in `Model Files/`
- 3. Update the corresponding `feature_schema.json` version field
- 4. Restart the API server

Section 5: Retraining Strategy

5.1 Retraining Triggers

Retraining should be initiated when **any** of the following thresholds are crossed:

Trigger	Threshold	Measurement Method
Feature drift (any single feature)	Drift score > 0.2 (Wasserstein) or p-value < 0.01 (chi-sq)	Notebook 2 / Evidently
Target label distribution drift	Chi-squared p-value < 0.05	Notebook 2 / TargetDriftPreset
> 30% of model features drifting simultaneously	Feature count threshold	Notebook 2 summary table
Production performance degradation	F1-macro drops > 10% from baseline	Requires ground-truth labels (delayed)
Schema mismatch (new city/provider)	HTTP 422 rate > 1% in 24h	API log monitoring

5.2 Retraining Checklist

When a trigger fires:

- ☐ Collect new labelled data covering the drift period
- ☐ Re-run `Phase 2 / Build Features.ipynb` on the expanded dataset
- ☐ Re-run relevant Phase 3 training notebook (`Risk Model.ipynb` or `Claim Outcome Model.ipynb`)
- ☐ Evaluate on a held-out test set — confirm metrics meet or exceed v1.0 baseline
- ☐ Update `feature_schema.json` if new categories or ranges apply
- ☐ Increment model version string in `Api.py`
- ☐ Replace `.pkl` in `Model Files/`
- ☐ Restart the API and run smoke tests (`/health` , `/predict` with known inputs)
- ☐ Archive old `.pkl` with its version tag
- ☐ Update Phase 4 model card notebooks with new evaluation results

5.3 Version Bumping Convention

Change Type	Version Bump	Example
New training data, same features and algorithm	Patch	1.0 → 1.1
New features added or removed	Minor	1.0 → 1.1
Algorithm change or complete redesign	Major	1.0 → 2.0

Note: Major version bumps require updating API documentation and client code if input schema changes.

Section 6: Compliance Notes

6.1 Data Minimisation

- Prediction logs contain only a SHA-256 hash of input features — no raw patient identifiers, ages, or financial figures are persisted in logs
- The API does not store or cache request payloads beyond the current request lifecycle
- Patient IDs are never accepted as API inputs

6.2 Fairness Monitoring

The following fairness dimensions should be evaluated before any production deployment:

Dimension	Relevant Feature	Recommended Check
Gender bias	gender (M/F)	Compare F1-macro per class by gender subgroup
Geographic bias	city	Compare rejection rate predictions by city
Age bias	age (claim model)	Slice performance by age brackets (<18, 18–60, >60)

6.3 Explainability

SHAP feature importance analysis is documented in **Phase 4** notebooks:

- Phase 4 / 1 – Risk Model Evaluation Explanation Model Card.ipynb
- Phase 4 / 2 – Claim Model Evaluation Explanation Model Card.ipynb

The API currently returns a confidence score but does not return per-feature attributions. If a clinician or claims adjuster requires an explanation, the SHA-256 hash in the log can be used to match a specific prediction to its input features (provided the original payload is retained by the calling system).

6.4 Intended and Prohibited Uses

Intended Use	Prohibited Use
Clinical triage prioritisation support tool	Sole basis for patient treatment decisions

Intended Use	Prohibited Use
Claims processing workflow automation	Automatic rejection of claims without human review
Operational resource planning	Denial of care or insurance coverage

6.5 Model Cards Cross-Reference

Full model cards (training data, evaluation metrics, fairness analysis, limitations) are in:

- [Phase 4 / Risk Model Evaluation Explanation Model Card.ipynb](#)
- [Phase 4 / Claim Model Evaluation Explanation Model Card.ipynb](#)

```
In [1]: # Programmatically verify governance artifacts exist
from pathlib import Path

REPO_ROOT = Path(".").resolve().parent.parent
PHASE6 = Path(".").resolve()

artifacts = {
    "patient_risk_model.pkl": REPO_ROOT / "Model Files" / "patient_risk_model.pkl",
    "claim_outcome_model.pkl": REPO_ROOT / "Model Files" / "claim_outcome_model.pkl",
    "patient_risk_schema.json": REPO_ROOT / "Model Files" / "patient_risk_model_feature_schema.json",
    "claim_outcome_schema.json": REPO_ROOT / "Model Files" / "claim_outcome_model_feature_schema.json",
    "validate.py": PHASE6 / "validate.py",
    "Api.py": REPO_ROOT / "Notebooks" / "Phase 5 – Deployment and API Integration" / "Api.py",
    "predictions.log": REPO_ROOT / "Notebooks" / "Phase 5 – Deployment and API Integration" / "logs" / "predictions.log",
    "drift_report (risk)": PHASE6 / "patient_risk_drift_report.html",
    "drift_report (claim)": PHASE6 / "claim_outcome_drift_report.html",
}

print(f"{'Artifact':<35} {'Status':<10}")
print("-" * 45)
all_present = True
for name, path in artifacts.items():
    status = "✓ Present" if path.exists() else "✗ Missing"
    if not path.exists():
        all_present = False
    print(f"{name:<35} {status}")

print()
if all_present:
    print("All governance artifacts are present.")
else:
    print("Some artifacts are missing – run the relevant notebooks to generate them.")
```


Artifact	Status
patient_risk_model.pkl	✓ Present
claim_outcome_model.pkl	✓ Present
patient_risk_schema.json	✓ Present
claim_outcome_schema.json	✓ Present
validate.py	✓ Present
Api.py	✓ Present
predictions.log	✓ Present
drift_report (risk)	✗ Missing
drift_report (claim)	✗ Missing

Some artifacts are missing – run the relevant notebooks to generate them.